本机作品档案 · MACOS · 2026

用 pi 与 dsh 造出来的东西

过去半年，我把需求用中文说给两个跑在本机的编程助手——pi 和 dsh—— 由它们读仓库、改代码、跑测试、打包、装进 /Applications。 这里是我让它们造出来的 12 件软件与工具、一组 Agent 工具链，以及每件产物背后的构建轨迹。

12 件成品	15.5 万行 Rust + JS/TS
258 次构建会话	4 已装进 /Applications
8 GitHub 公开仓库	

01 桌面应用 Rust · GPUI · 原生

GitHub ↗

treq

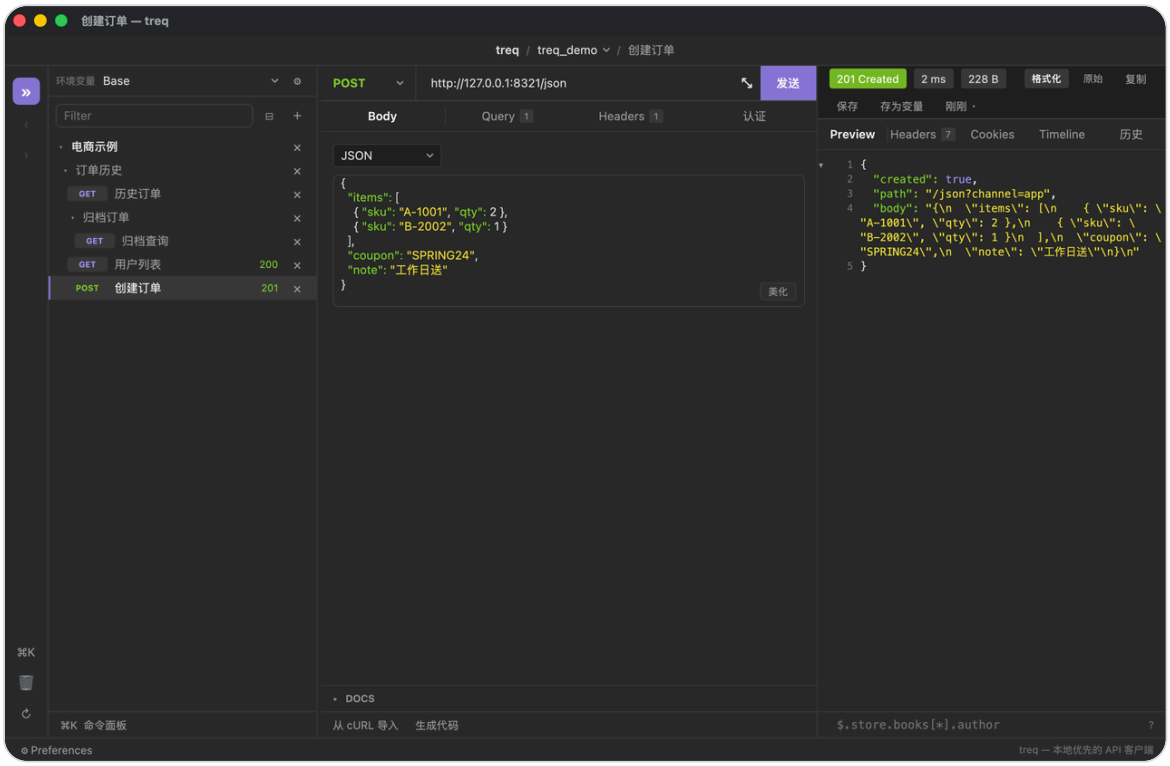
API CLIENT · RUST + GPUI

本地优先的接口调试客户端：界面像 Insomnia，数据是磁盘上一堆能进 Git 的 YAML。

左边是空间 → 集合 → 分组（任意层级）→ 请求的树，中间编辑，右边看响应。 五种 Body、四种认证、JSON 高亮与折叠、状态码 / 耗时 / 大小、JSON 路径提示、cURL 导入与代码生成。**285 个测试函数**和每次改动后的截图自检，是它敢连续重构的前提。

33.3k 行 Rust · 21 commits · 已装 /Applications/treq.app

- 构建轨迹：pi 5 个会话（71 MB 记录）· README 自述「绝大部分代码由 Claude Code / pi 编写」



maildesk

GitHub ↗

MAIL CLIENT · RUST + GPUI

原生 macOS 邮件客户端：没有 WebView，没有现成邮件库，IMAP/SMTP/MIME 全部手写。

它只回答一个问题：一个邮件客户端能有多小。3.6 万封邮件实测 **94 MB 物理内存、空闲 0.3% CPU、5.4 MB 二进制**。 HTML 正文用真实文本排版渲染——标题、列表、表格、引用回复、内联图片，还能拖选复制。

28.8k 行 Rust · 51 commits · 10-02 → 10-16 · 已装 /Applications

- 构建轨迹：dsh 9 个会话（含早期 email-desktop 目录）

artifeeds

GitHub ↗

FEED READER · RUST + GPUI

为「上千个源、又快又省」写的桌面 RSS / Atom / JSON Feed 阅读器。

单定时器调度、两级预算并发、条件请求（没更新就只花一个 304、零字节正文）、自适应轮询，每个队列都有硬上限。 三种浏览维度（按博客 / 标签 / 分类）全部虚拟化。 还能让本机 pi 把任意网页变成订阅：一个 OPML 落地 **102 个源 / 12 个分组**。

18.0k 行 Rust · 已装 /Applications · pi 集成

- 构建轨迹：pi 项目记忆 + dsh 2 个会话

TVA

VOICE AGENT · RUST + GPUI

按住一个键说话，本机的 pi / dsh / opencli 替你动手。

whisper.cpp 完全离线识别，**tiny 出实时预览、small 出终稿**（解码耗时 0.15s / 0.9s）。 一条指令拆成录音 → 识别 → 理解 → 思考 → 执行 → 完成六步，每步耗时都来自真实信号； 执行后端可一键切换 pi RPC / dsh headless / opencli / shell。

5.0k 行 Rust · 离线可用 · dist/TVA.app

- 它是 pi / dsh 的前端而非产物：本机会话库里没有它的构建记录，按实际来源标注

02 本地服务与内容 axum · SQLite · 单文件

toval

GitHub ↗

WRITING UNIVERSE · 38.3K 行

个人创作宇宙工作台：故事、角色、事件、章节都长在一张图上。

universe → entity / era / event / story → chapter 的 SQLite 图模型；AI 对话产出提案，落库前留修订快照； 服务端渲染 + HTMX，静态样式提前构建，运行期不需要 Node；一个二进制，只监听 127.0.0.1。

38.3k 行 Rust · 单文件二进制 · DeepSeek 生成

- 构建轨迹：dsh **20 个会话**——本机最密集的一个项目

tblogs

BLOG SYSTEM · 两段式

拆成两半的博客系统：本地写作，服务器只读。

`tblogs-admin` 读本地 Markdown → 生成内容包与部署文件 → 发布； `tblogs-server` 只提供 HTTP，**没有任何写作入口**，被攻破也改不了文章。 axum + SQLite + askama + pulldown-cmark，无前端构建、无 CDN。

7.7k 行 Rust · 10 commits · systemd + Caddy/nginx

- 构建轨迹：dsh 3 个会话 · deploy/ 含 cloudflared 与 DDNS 定时器

gov-reports

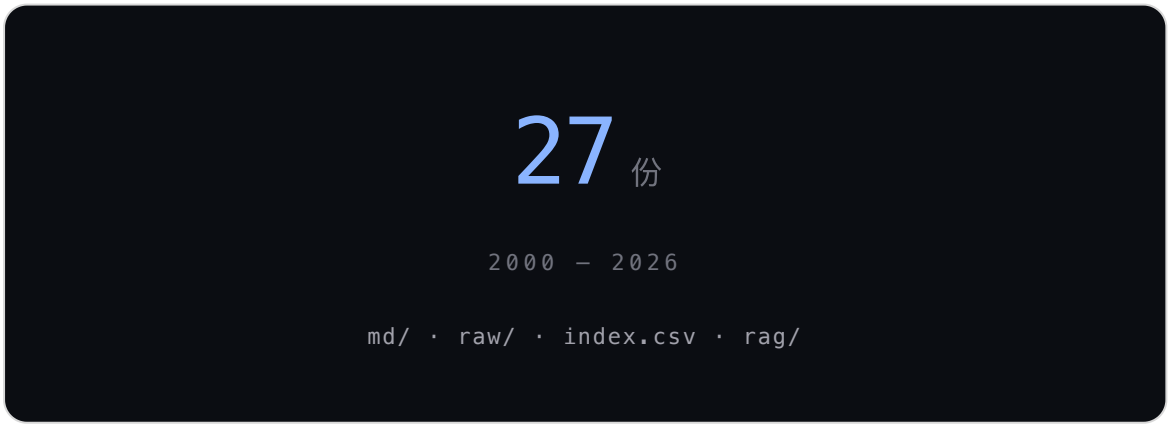
CORPUS + RAG · 27 年

27 年《政府工作报告》全文语料库，外加一层检索。

2000–2026 无缺年，**27 份 Markdown、约 50.4 万字**，每年一份元信息表 + 正文；保留原始 HTML 快照以便离线复核。 中央站点改版后缺失的 6 年，用省级政府官方转载补齐并逐条标注来源。`rag/` 里是建库、嵌入、检索三件套。

27 份 · 50.4 万字 · 原始 HTML 快照 · 可重跑

- 构建轨迹：dsh 3 个会话 · 抓取脚本 fetch_reports.py + 三级兜底来源



03 设备与系统小工具

贴着 macOS / 浏览器 / uTools 的边角

GitHub ↗

RQ

RaptorQR · Host

SCREEN QR TRANSFER · RUST

用屏幕上的动画二维码，在没网、没共享剪贴板的两台机器之间搬文本和文件。

上游是浏览器发、摄像头收；这个 fork 新增 **host/**：不起摄像头，直接扫屏幕一块区域—— 专治「隔离虚拟机 ↔ 宿主机」。macOS 是 gpui 窗口版（实时预览、拖框选、多显示器、系统通知）， Windows 是无界面控制台版。**收到的文本不落盘，直接进剪贴板**；文件写 ~/Downloads。

`host 16.8k 行 Rust` · `93 commits` · `universal 二进制` · `已装 /Applications`

- 构建轨迹：pi 3 + dsh 2 个会话 · GitHub Actions 出 Windows 构建



GitHub ↗

TBAS

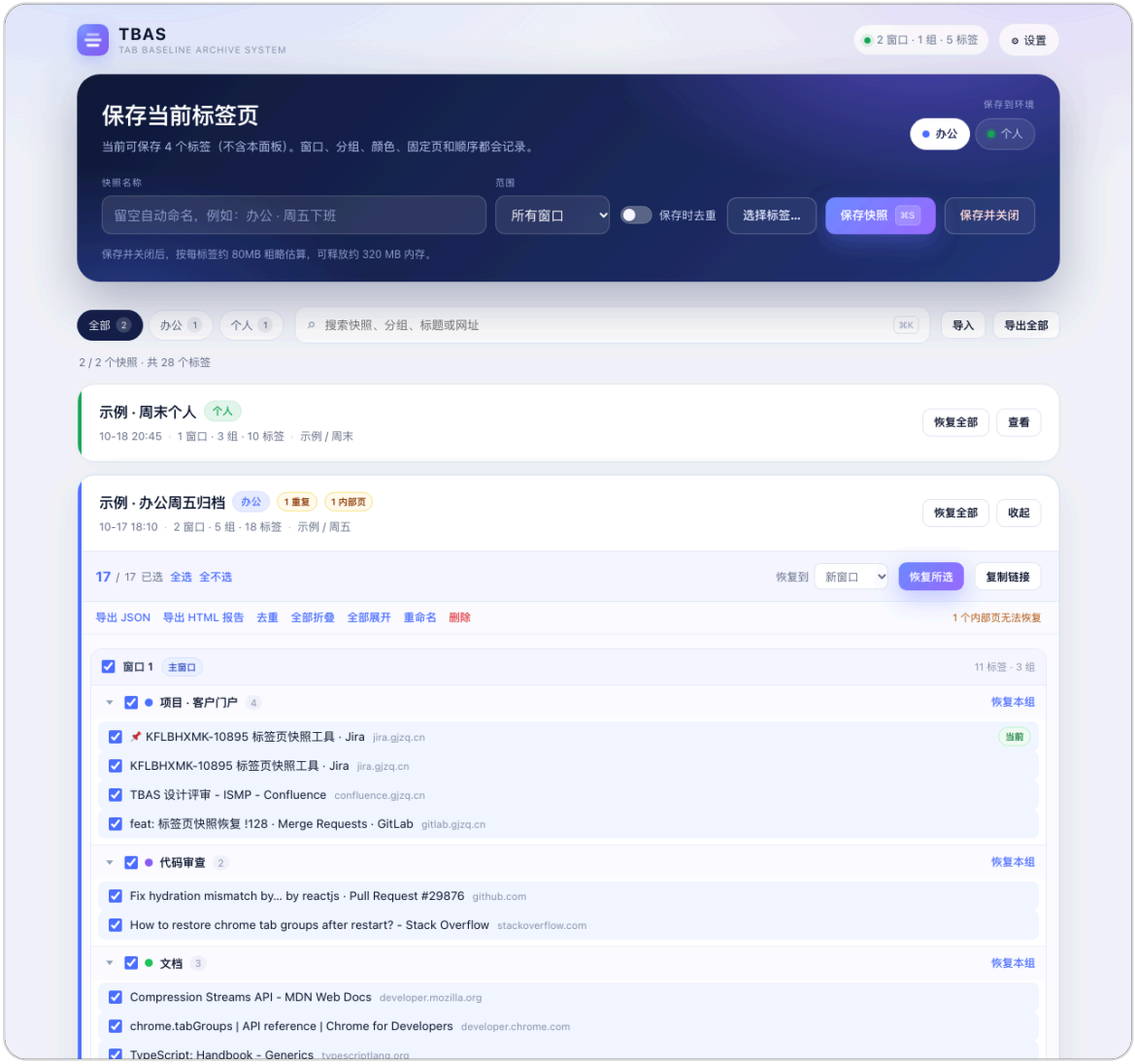
TAB BASELINE ARCHIVE · NODE + EDGE 扩展

把 Edge 里越开越多的分组标签页，存成可检索、可选择性恢复、可安全关闭的快照。

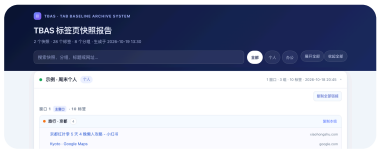
记录窗口、分组、颜色、折叠状态、固定页、顺序和当前标签，按「办公 / 个人」归档；保存并关闭可释放几百 MB 内存； 还能导出离线 HTML 报告。保存时去重、识别 utm 跟踪参数。自带 4 套单测 + 端到端。

5.3k 行 JS · CLI + 扩展 · 4 套测试 + e2e

- 构建轨迹：dsh 1 个会话 · 截图即 docs/ 下三张实拍









fd

utools-fd-search

UTOOLS PLUGIN · FD + RG + FZF

GitHub ↗

文件名和文件内容一起搜：三个命令行工具拼成一个 uTools 插件。

字面量 / glob / 模糊三种模式；类型、时间过滤；结果流式渲染，↑↓ 选、Enter 打开、⌘Enter 定位。 可选接本机 pi / dsh，把「上周改过的资产配置表格」解析成查询条件。

1.5k 行 JS/HTML · 38 KB .upxs

- 构建轨迹：pi 4 个会话 · 三个依赖都可降级

cm

utools-selfie

UTOOLS PLUGIN · CAMERA

GitHub ↗

在 uTools 里录摄像头：用一个脚本 app 把 ffmpeg 变成独立的 TCC 实体。

uTools 有硬化运行时却缺相机权限，getUserMedia 拿不到帧、子进程也被 TCC 拒绝。 解法是 CamHelper.app 经 LaunchServices 启动：预览进程常驻、录制进程按需，分段 MP4 可安全强杀。

9 KB .upxs · 预览 + 录制双进程

- 构建轨迹：pi 1 个会话（约 1 MB 记录）

p2

pdf2str

CLI · 58 行

把 PDF 转成纯文本的一行命令，扫描件走 OCR。

基于 liteparse：`--ocr`、`--pages 1-5,10`、`--lang chi_sim+eng`。 小到只有 58 行，是「让 agent 顺手写个工具」的最小样本。

58 行 Rust · liteparse

- 构建轨迹：RustroverProjects/pdf2str

04 Agent 工具链 给人和给 agent 用的那部分

01 **dsh-remote / win-task**

跨机派单：任务走 ssh 的 stdin，远端 Windows 上的 dsh headless 执行，中文、引号、换行都不被 shell 转义破坏。

02 **ui-unwedge / ui-wakeguard**

macOS 窗口服务器失步（Dock 卡住、空间切不动）的诊断与逐级复位；唤醒探测用 `sysctl kern.waketime`，只有新唤醒才做重诊断。

03 **dlp-toggle**

终端管控（联软 / 微步）的临时开关：用 `bootout` 只卸载当前会话，重启自动恢复。

04 **wb-otel-env**

从登录快照读账号，注入 WorkBuddy 的 OTel 上报环境变量，由 LaunchAgent 开机执行。

05 **deepseek-balance.ts**

pi 扩展：会话启动即拉余额，之后每 5 分钟刷新到状态栏——把「还剩多少钱」变成常驻信息。

06 **13 个自建 skill**

把重复的流程固化成可被 agent 调用的说明书：Jira 三件套、ISMP 审计、会议室预约、macOS 三件套、成本审计……

`dashi-ppt`

按预置视觉主题生成可离线打开、可在浏览器编辑的 HTML 演示，导出 PPTX / PDF

`jira-issue-chain-create`

按模板在 jira.gjzq.cn 建「需求→评审→特性→故事→子任务」五层单链

`jira-cmmi-daily-check`

读 CMMI 不符合项清单，按规则逐条改单并回读验证

`jira-version-create`

按日期生成 ISMP 的「故事 / 特性」两个发布版本

`ismp-account-role-export`

把 ISMP 导出的角色-菜单表加工成域账户审计清单

`eboss-meeting-room-book`

查空闲会议室并预约 / 改约，用 opencli 驱动 Edge

`broker-chat-compliance-check`

用本地 DeepSeek 对券商坐席聊天记录做合规质检

`macos-windowserver-wedge-triage`

UI 卡死的分诊与恢复流程

`macos-modern-cli-shell-setup`

装齐现代 CLI 替代品并配好 zsh alias 与全局 AGENTS.md

`mem-skill-macos`

「系统数据」膨胀的定位与安全清理，按可回收量分级

`pi-llm-cost-audit`

从 session jsonl 聚合每日 / 每会话 token 与花费

`split-large-excel-to-zip`

大 Excel（含加密）按固定行数拆分并打包

`iterm2-cmdv-image-paste`

让 ⌘V 在 TUI 里既能粘文本又能贴图，只改按键映射

05 它们是怎么被造出来的

说需求 → 验收，而不是写代码

01

说需求

一句话、一段抱怨、一张截图，不写规格

02

读仓库

agent 先读 README、测试与现有代码，形成约束

03

小步改

一次一件事，跑测试、跑截图自检

04

装上去

bundle → /Applications，真实用一轮

05

回写文档

把取舍写进 README / AGENTS.md，下次继承

pi

TERMINAL AGENT

- 常驻终端，`--mode rpc` 可被别的程序调用（TVA 就是这么接的）
- 项目记忆 + 全局 AGENTS.md，跨会话继承约束
- 扩展生态：余额、终端联动
- 偏向「在一个仓库里长时间磨一个东西」

```
190 个会话 · 592 MB 记录
```

dsh

DESKTOP + WEB AGENT

- 桌面 / Web / headless 多形态，可远程 SSH 部署
- 任务板、子代理、技能、插件市场
- 长任务与多项目并行，会话可按工作区分组
- 偏向「把一整套流程跑完」

```
68 个会话 · 109 MB 记录
```

06 口径与证据

哪些是数出来的，哪些是推出来的

代码行数

`src/` 与 `crates/` 下 `.rs` 的行数合计，不含 `target/` 与依赖；JS/TS 同理。共 154,694 行。

会话数

pi：`~/.pi/agent/sessions/<工作区>/*.jsonl` 共 190 个；dsh：`~/.dsh/sessions/<工作区>/` 共 68 个，合计 258。目录名即当时的工作目录。

装机

以 `/Applications` 下实际存在为准：treq、maildesk、artifeeds、RaptorQR Receiver。

公开仓库

以 `git remote` 指向 `github.com/tuean` 为准：treq、maildesk、RaptorQR、utools-fd-search、toval、artifeeds、tbas、utools-selfie。

归属

会话目录只能说明「在那个项目里工作过」，不等于逐行归属。treq 的 README 有作者自述作为直接证据，其余按会话密度与提交节奏推断。

例外

TVA 驱动 pi / dsh，但本机会话库里没有它的构建记录，因此单独标注，不算作 pi/dsh 产物。

时间

提交时间与文件时间取自 git / 文件系统；最早的 RaptorQR 提交在 2026-05，最近的在 2026-10-16。

截图

全部来自各仓库 `docs/`，非本次补拍；treq、TBAS 为运行实拍。

彩蛋：dsh 桌面宠物「D老师」· 亲密度 484 · 396 轮